3

Unit 3: Combinational Logic Circuits

Digital Logic Design

CHANDRAS EDU

Never stop learning

Outline

3.1 Combinational Circuit Analysis & Design
3.2 Binary Adder & Subtractor
3.3 BCD Adder & Binary Multiplier
3.4 Magnitude Comparator
3.5 Decoders & Encoders
3.6 Multiplexers & Demultiplexers

3.1 Combinational Circuit Analysis & Design

IN SIMPLE TERMS

Analysis goes from circuit to function; design goes from a word description to a truth table, a simplified function and finally the circuit.

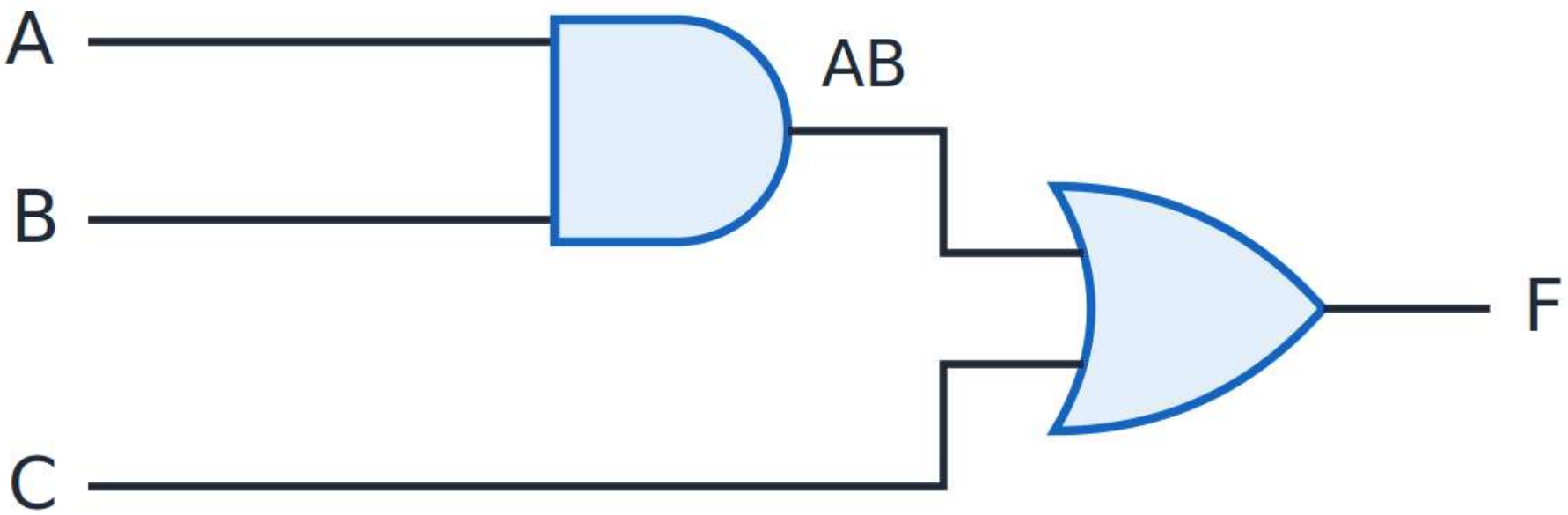
Combinational circuits

- A **combinational circuit** has outputs that depend only on its **present** inputs: never on past inputs and never on a clock
- n binary inputs go in and m binary outputs come out
- Every output is a Boolean function of the inputs, built from logic gates with **no feedback loop**
- Adders, comparators, decoders and multiplexers in this unit are all combinational
- Unit 4 is where memory enters the picture

Analysis procedure

- **Analysis** starts from a circuit diagram and recovers the Boolean function it computes
- **1. Label every wire:** name the output of every gate, from the inputs toward the outputs, not just the final output
- **2. Write each gate's output** as a Boolean expression in terms of the wires feeding it; substitute until only the circuit inputs remain
- **3. Build a truth table** (optional but recommended) by evaluating the final expression for every input combination

Analysis example circuit



Circuit for the analysis example: F = AB + C.

Worked example: analysing a 2-gate circuit


```
Step 1 — label the AND gate's output:   w1 = AB
Step 2 — the OR gate combines w1 and C:   F = w1 + C
Step 3 — substitute:   F = AB + C
```

- Evaluating F = AB + C for all 8 input combinations gives the truth table
- F is 1 whenever C is 1 (regardless of A, B), or whenever both A and B are 1

Design procedure

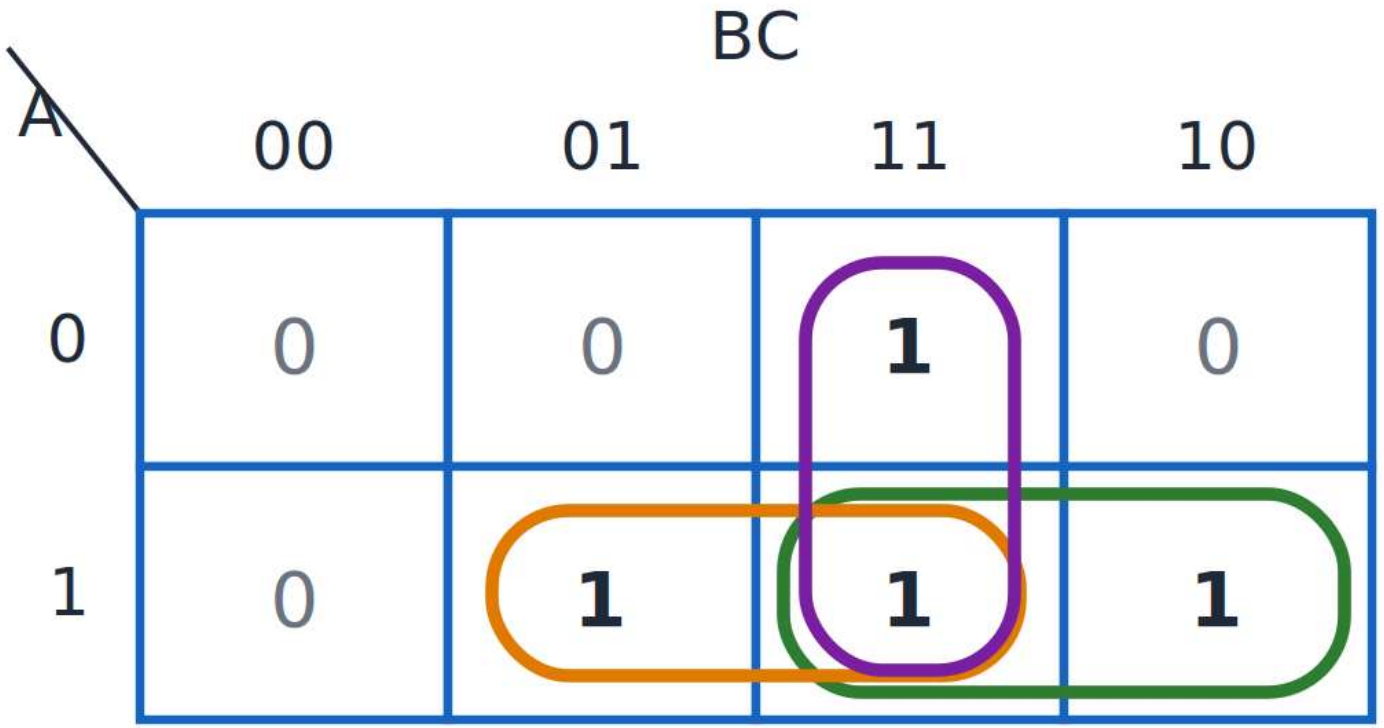
- **Design** starts from a written specification and produces a minimal circuit
- **1.** State the problem precisely; identify the number of inputs and outputs
- **2.** Assign letter symbols to each input and output variable
- **3.** Derive the truth table
- **4.** Obtain the minimal Boolean function for each output, usually with a K-map (Unit 2)
- **5.** Draw the logic diagram from the minimal SOP or POS expression and verify it against the specification

Worked example: 3-input majority function


```
Specification: inputs A, B, C; output F
F = 1 whenever a majority (2 or more) of the inputs are 1
Truth table → minterms: m3 (011), m5 (101), m6 (110), m7 (111)
F(A,B,C) = Σm(3,5,6,7)
```

- F is 1 for the input combinations with two or three 1s

Majority function: K-map



m6 + m7 → AB **m5 + m7 → AC** **m3 + m7 → BC**

K-map for the majority function: F = AB + AC + BC.

Majority function: minimal SOP and circuit


```
Pair {m6, m7}   (A=1, B=1, C varies):   term = AB
Pair {m5, m7}   (A=1, C=1, B varies):   term = AC
Pair {m3, m7}   (B=1, C=1, A varies):   term = BC
F = AB + AC + BC
```

- The three pairs share cell m7: a normal overlap, since a group only needs to cover a 1
- Circuit: three 2-input AND gates (one per term) feeding a 3-input OR gate, or two cascaded 2-input ORs: five gates in total
- No further simplification: every group is already a maximal pair

3.2 Binary Adder & Subtractor

IN SIMPLE TERMS

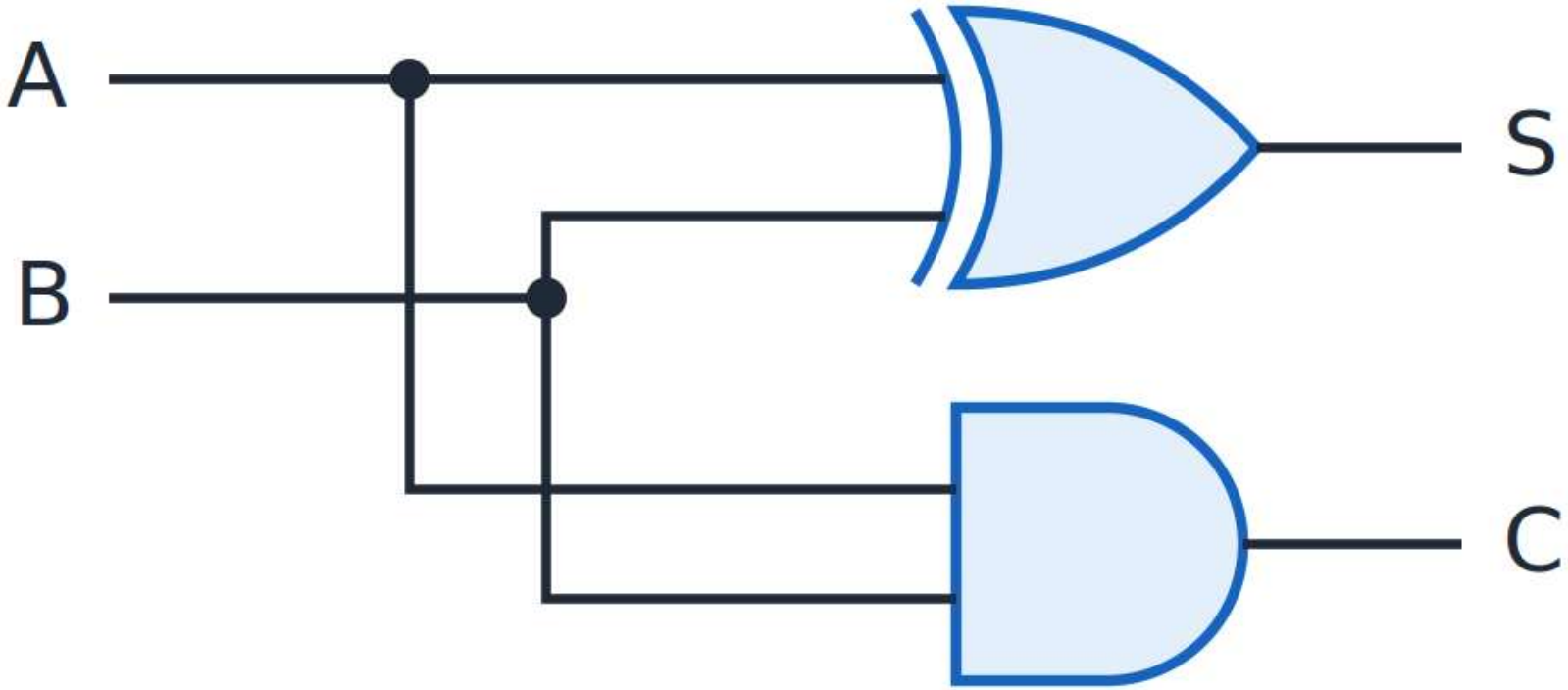
A half adder adds two bits; a full adder also adds the incoming carry; chain full adders to add whole numbers, and an XOR per bit lets the same circuit subtract.

Half adder

S = A ⊕ B **C = AB**. A half adder takes two bits and has no way to accept a carry-in.

A	B	S (Sum)	C (Carry)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half adder



Half adder: S = A ⊕ B, C = AB.

Full adder

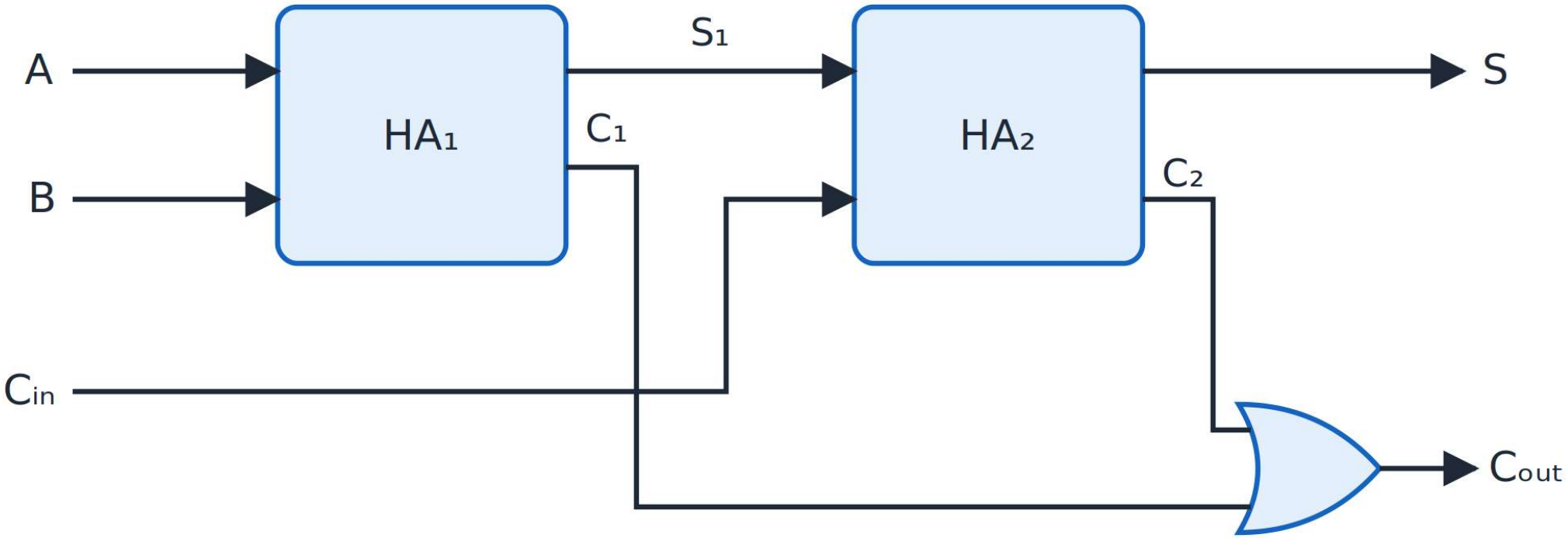
$S = A \oplus B \oplus C_{in}$ $C_{out} = AB + BC_{in} + AC_{in}$

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full adder from two half adders

- Reuse two half adders plus one OR gate rather than building the SOP expressions with fresh gates
- **HA_1** adds A and B: partial sum S_1 and carry C_1
- **HA_2** adds S_1 and C_{in}: final sum S and a second carry C_2
- C_1 and C_2 can never both be 1, because S_1 is a single bit
- OR-ing them gives C_{out} without a 3-input gate

Full adder from two half adders

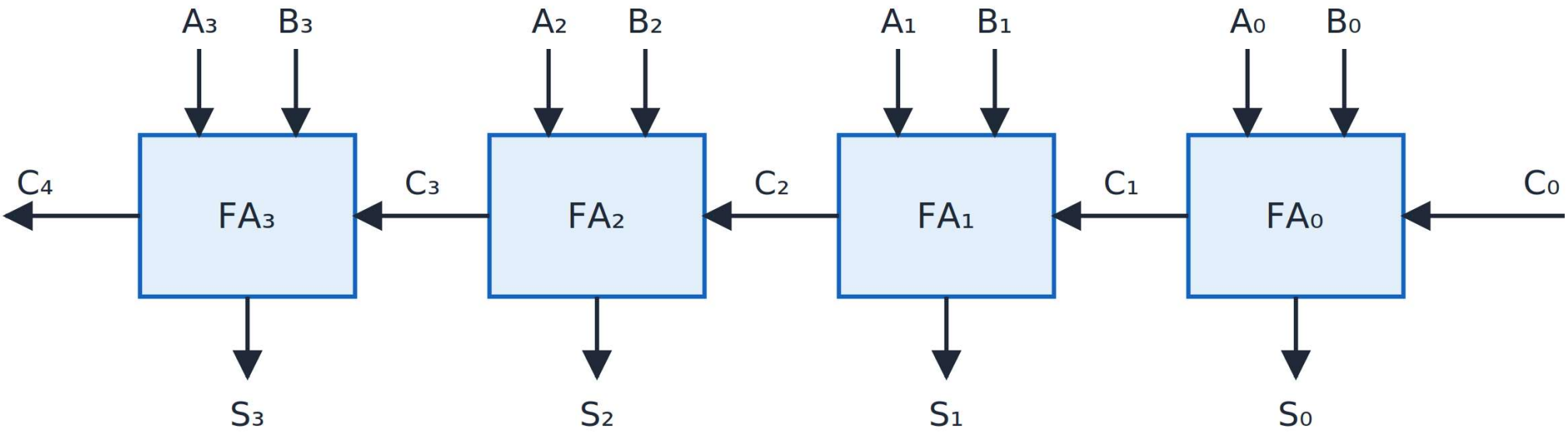


Full adder from two half adders and an OR gate. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 4.8.

Ripple-carry binary adder

- Chain **n full adders**: each stage's C_{out} feeds the next stage's C_{in}
- The result adds two **n-bit** binary numbers
- The carry "ripples" from the least significant bit to the most significant, one full-adder delay at a time
- **Drawback:** the most significant sum bit is not valid until the carry has passed through every stage

4-bit ripple-carry adder



Four-bit ripple-carry adder. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 4.9.

Worked example: 4-bit addition, 0110 (6) + 0011 (3)

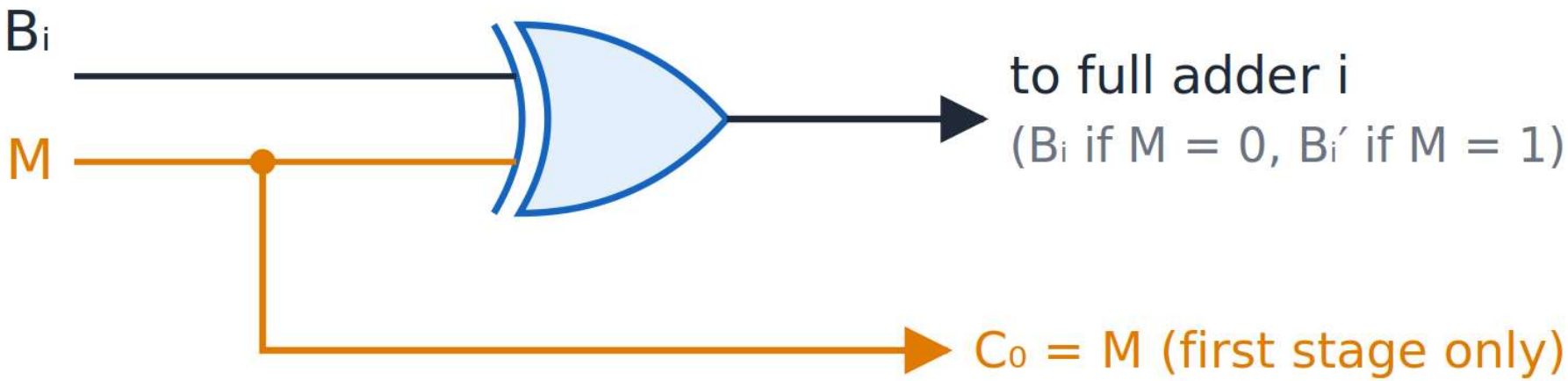

```
Bit position:   3 2 1 0
A            :  0 1 1 0
B            :  0 0 1 1
carry-in     :  1 1 0 0   (C3 C2 C1 C0)
-------------------------
Sum          :  1 0 0 1
carry-out    :  0 1 1 0   (C4 C3 C2 C1)
```

- Result: S = 1001 (9), C_4 = 0: no overflow. 6 + 3 = 9 ✓

Binary subtractor

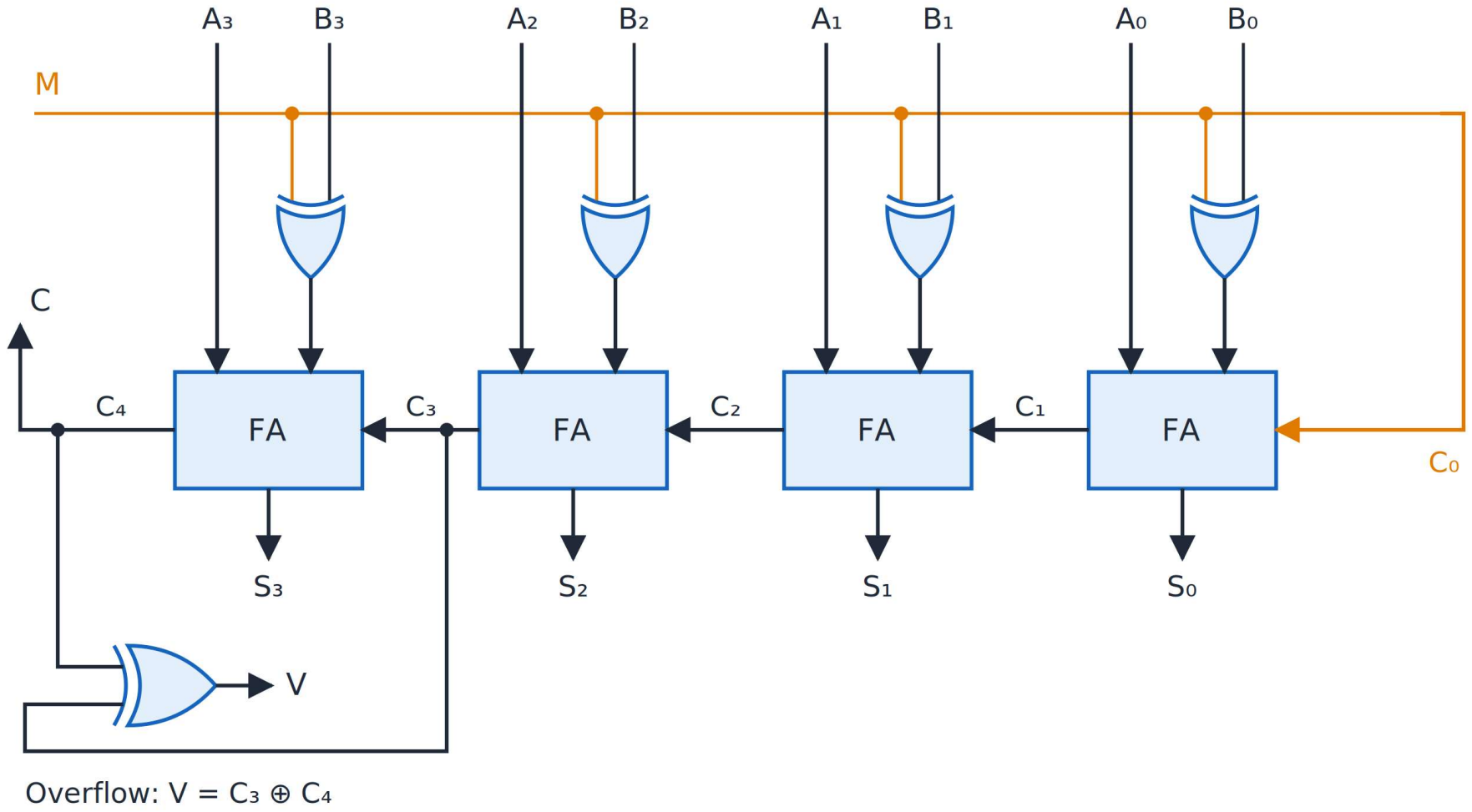
- Subtraction A − B is done by adding A to the **2's complement** of B (Unit 1)
- Add one control line **M** (0 = add, 1 = subtract) and one **XOR gate per bit of B**
- **M = 0:** each XOR passes B_i unchanged and C_{in} = 0: the circuit computes A + B
- **M = 1:** each XOR inverts B_i and C_{in} = 1 supplies the "+1": A + B′ + 1 = A − B
- All XOR gates share the same M line, and C_{in} of the adder is also tied to M

One XOR gate per bit of B



One XOR gate per bit of B: M = 0 passes B_i, M = 1 inverts it; M is also C_0.

Four-bit adder-subtractor



Four-bit adder-subtractor: M = 0 adds, M = 1 subtracts; $V = C_3 \oplus C_4$ flags overflow. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 4.13.

3.3 BCD Adder & Binary Multiplier

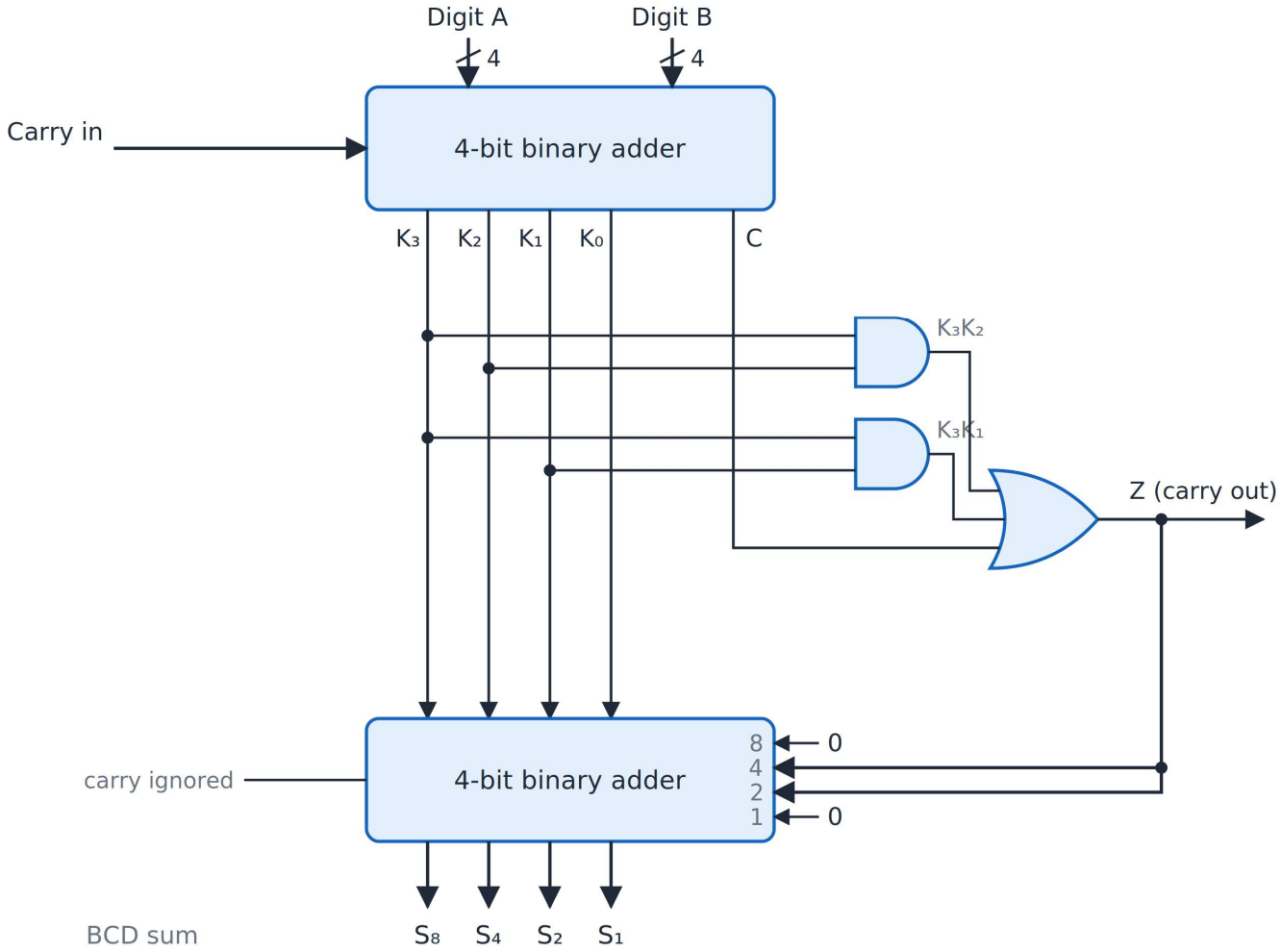
IN SIMPLE TERMS

Add BCD digits in binary, then add 6 when the result is over 9; multiplying bits is just AND, followed by adders for the partial products.

BCD adder: the correction rule

- **BCD** codes each decimal digit 0–9 as 0000–1001; patterns 1010–1111 never occur
- Adding two BCD digits in a 4-bit binary adder works only if the result is 0–9; 9 + 9 = 18 is 10010, not valid BCD
- Feed both digits (plus any carry-in) into a binary adder: raw sum **K** (0–19) and carry **C**
- If **K ≤ 9 and C = 0**: already valid BCD, use as is
- If **K ≥ 10, or C = 1**: add **0110 (6)**; its carry-out becomes the carry into the next decade
- **$Z = C + K_3K_2 + K_3K_1$**; Z = 1 means “add 6”

BCD adder



BCD adder: $Z = C + K_3K_2 + K_3K_1$ adds 0110 when the binary sum exceeds 9. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 4.14.

Worked example: 8 + 7 in BCD

$A = 1000\ (8),\quad B = 0111\ (7),\quad C_{in} = 0$

Binary adder: $1000 + 0111 = 1111 \rightarrow K = 1111\ (15),\ C = 0$

$K_3K_2 = 1 \cdot 1 = 1 \rightarrow Z = C + K_3K_2 + K_3K_1 = 0 + 1 + 1 = 1$ (add 6)

Correction adder: $1111 + 0110 = 1\ 0101 \rightarrow S = 0101\ (5),\ C_{out} = 1$

- Result: S = 5 with C_{out} = 1 into the next decade → “15” ✓ (8 + 7 = 15)

Binary multiplier

- Binary multiplication works like decimal long multiplication, bit by bit
- Each multiplier bit gives one **partial product** row: the multiplicand ANDed with that bit, shifted left by its position
- The rows are summed to give the product
- Each partial-product bit is one **2-input AND gate**: 0×0 = 0, 0×1 = 0, 1×0 = 0, 1×1 = 1
- An m-bit × n-bit multiplier needs **m × n AND gates** plus an array of adders

2-bit × 2-bit multiplier: layout


```
          A1    A0
×         B1    B0
-------------------
      A1B0  A0B0      ← row 0 (×B0)
A1B1  A0B1            ← row 1 (×B1), shifted left 1
-------------------
 P3    P2    P1   P0
```

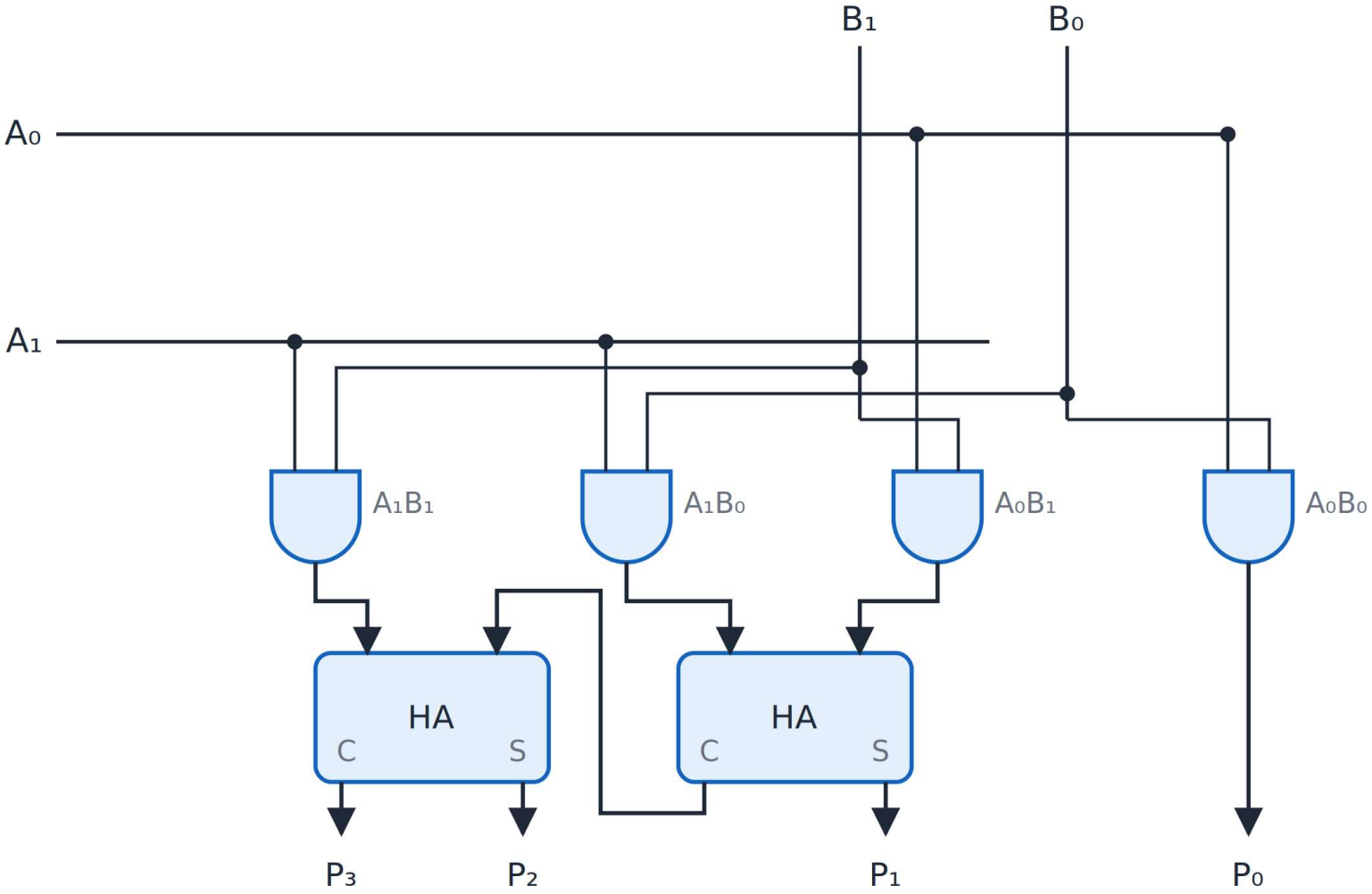
- **Column P_0:** A_0B_0 directly
- **Column P_1:** $A_1B_0 + A_0B_1$ → a half adder: sum = P_1, carry into the next column
- **Column P_2:** A_1B_1 + that carry → another half adder: sum = P_2, final carry = P_3

Worked example: A = 11 (3) × B = 11 (3)

$A_1B_0 = 1$, $A_0B_1 = 1 \rightarrow HA_1$: sum = 0, carry = 1 → $P_1 = 0$
$A_1B_1 = 1 \rightarrow HA_2$ adds 1 + 1 (carry):
sum = 0, carry = 1 → $P_2 = 0$, $P_3 = 1$
Product = 1001 (9) = 3 × 3 ✓

- Wider multipliers follow the same pattern: more AND gates and a bigger adder array
- Full adders are used once three or more bits land in the same column

Two-bit by two-bit binary multiplier



Two-bit by two-bit binary multiplier. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 4.15.

3.4 Magnitude Comparator

IN SIMPLE TERMS

Compare from the most significant bit down: the first bit where the numbers differ decides which is larger.

1-bit comparator

(A>B) = AB′ (A<B) = A′B (A=B) = A′B′ + AB = (A ⊕ B)′ (the XNOR of A and B). All three read off directly: no K-map needed.

A	B	A>B	A=B	A<B
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

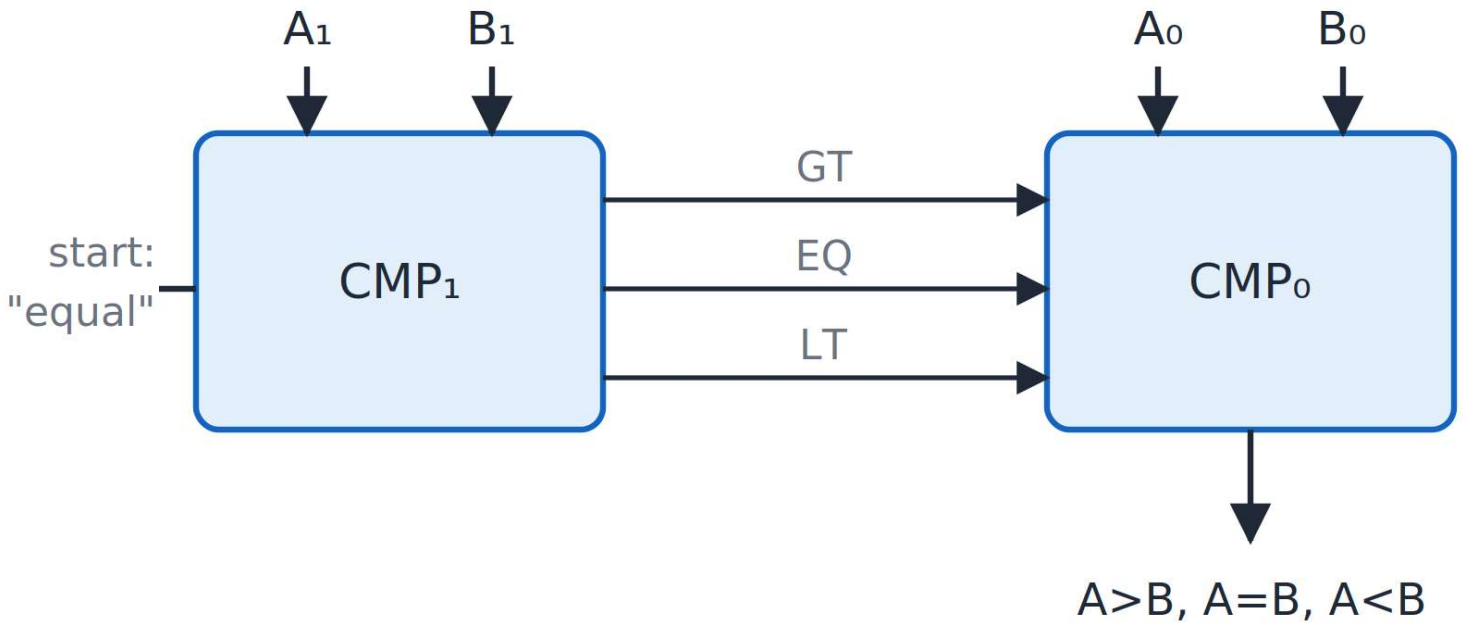
Cascading to n bits

- Comparison must start at the **most significant bit**: if the MSBs differ, no lower bit can change the outcome
- Only when a pair of bits is **equal** does the comparison continue to the next bit
- With $x_i = (A_i = B_i)$, for $A = A_1A_0$ against $B = B_1B_0$:
- $(A>B) = A_1B_1' + x_1 \cdot A_0B_0'$
- $(A<B) = A_1'B_1 + x_1 \cdot A_0'B_0$ $\quad$ $(A=B) = x_1 \cdot x_0$
- The pattern extends to any width

Two cascaded 1-bit stages

- In hardware: two 1-bit comparator stages cascaded, bit 1 (the higher-order bit) first
- **CMP_1** compares A_1 with B_1 and outputs (GT, EQ, LT); its EQ output is the x_1 signal
- If CMP_1 already decided (not EQ), **CMP_0** passes that decision through as the final answer
- Only if CMP_1 says EQ does CMP_0's own comparison of A_0 and B_0 decide
- This is exactly what "x_1 gates whether bit 0's comparison counts" means in circuit form

Two-bit comparison: two cascaded stages

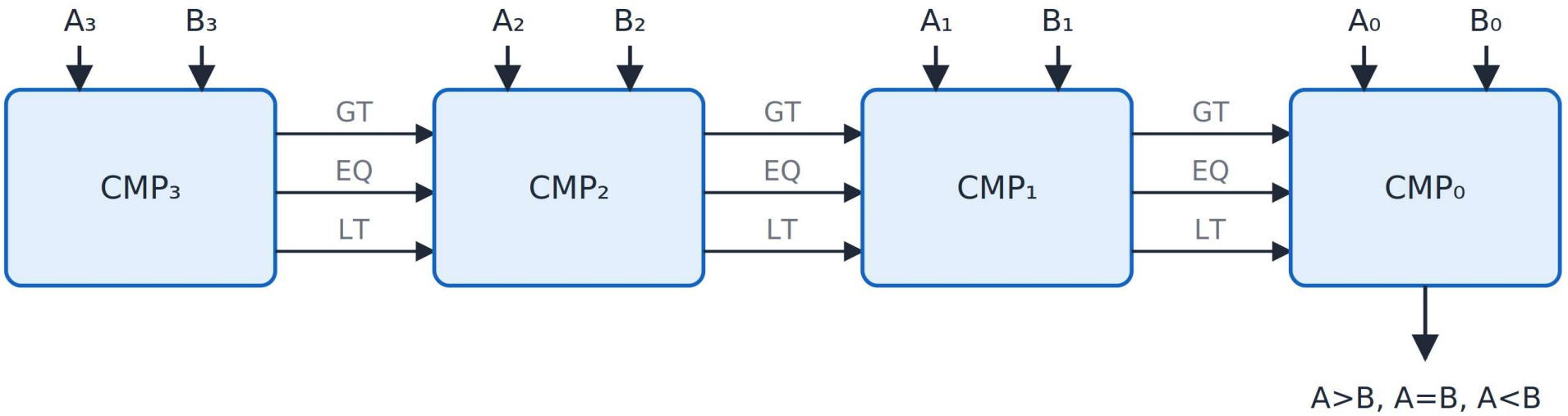


Two-bit comparison as two cascaded 1-bit stages, most significant bit first.

Chain of 1-bit stages

- Extend the idea to 4 bits: a chain of 1-bit stages, most significant bit first
- Each cascade line carries the **full running (GT, EQ, LT)** decision from every bit compared so far, not just a “still tied” flag
- The cascade entering CMP_3 is fixed at **(0, 1, 0)**: “equal so far”, vacuously true
- If the incoming cascade already says GT or LT, the stage **passes it straight through**
- Only if it says EQ does the stage’s own bit comparison update the result
- Only the final stage’s outputs are the overall result

Four-bit comparison: chain of stages



Four-bit comparison as a chain of 1-bit stages, each passing GT, EQ, LT to the next.

Worked example: A = 1011 (11), B = 1010 (10)


```
Bit 3: A3=1, B3=1 → equal, cascade "still equal" to bit 2
Bit 2: A2=0, B2=0 → equal, cascade "still equal" to bit 1
Bit 1: A1=1, B1=1 → equal, cascade "still equal" to bit 0
Bit 0: A0=1, B0=0 → A0 > B0, and every higher bit was tied
Result: A > B   (11 > 10) ✓
```

- Decided entirely by the least significant bit, because every bit above it matched

3.5 Decoders & Encoders

IN SIMPLE TERMS

A decoder turns a binary number into one active output line; an encoder does the reverse; a priority encoder picks the highest active input.

2-to-4 decoder

An **n-to-2^n decoder** has n inputs and up to 2^n outputs; exactly one output is 1. Each output is one minterm: $D_0 = A'B'$ $D_1 = A'B$ $D_2 = AB'$ $D_3 = AB$

A	B	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Enable input

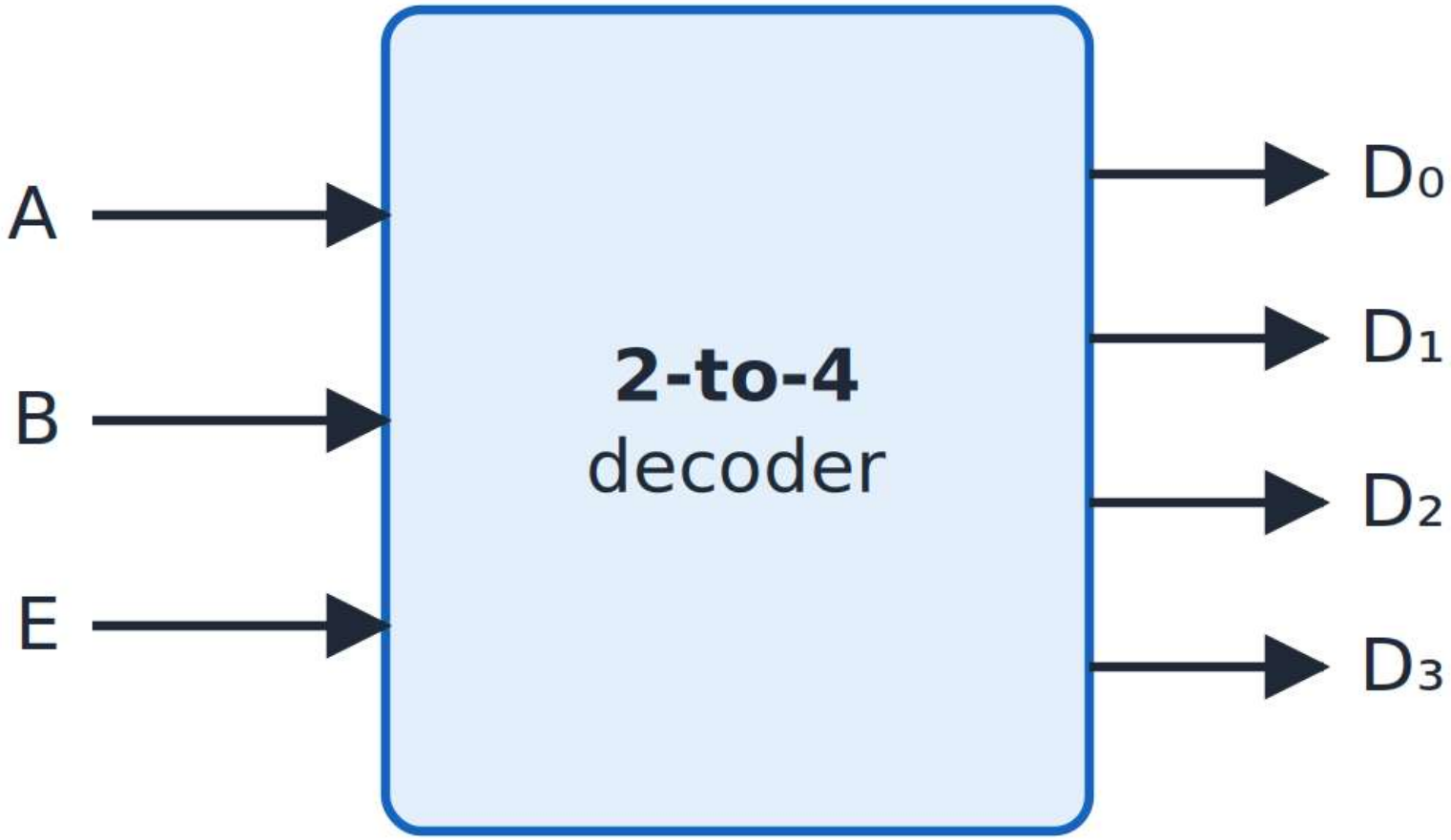
- Most real decoder chips add an **enable** input E
- Every output AND gate gets a third input tied to E
- All outputs stay 0 whenever E = 0, regardless of A and B
- This makes decoders useful as **chip-select logic**: only the addressed chip's enable line goes active
- The next slides show $D_2 = AB'E$ built out, then the whole decoder as a block

One decoder output with enable



One decoder output with enable: $D_2 = AB'E$.

2-to-4 decoder with enable

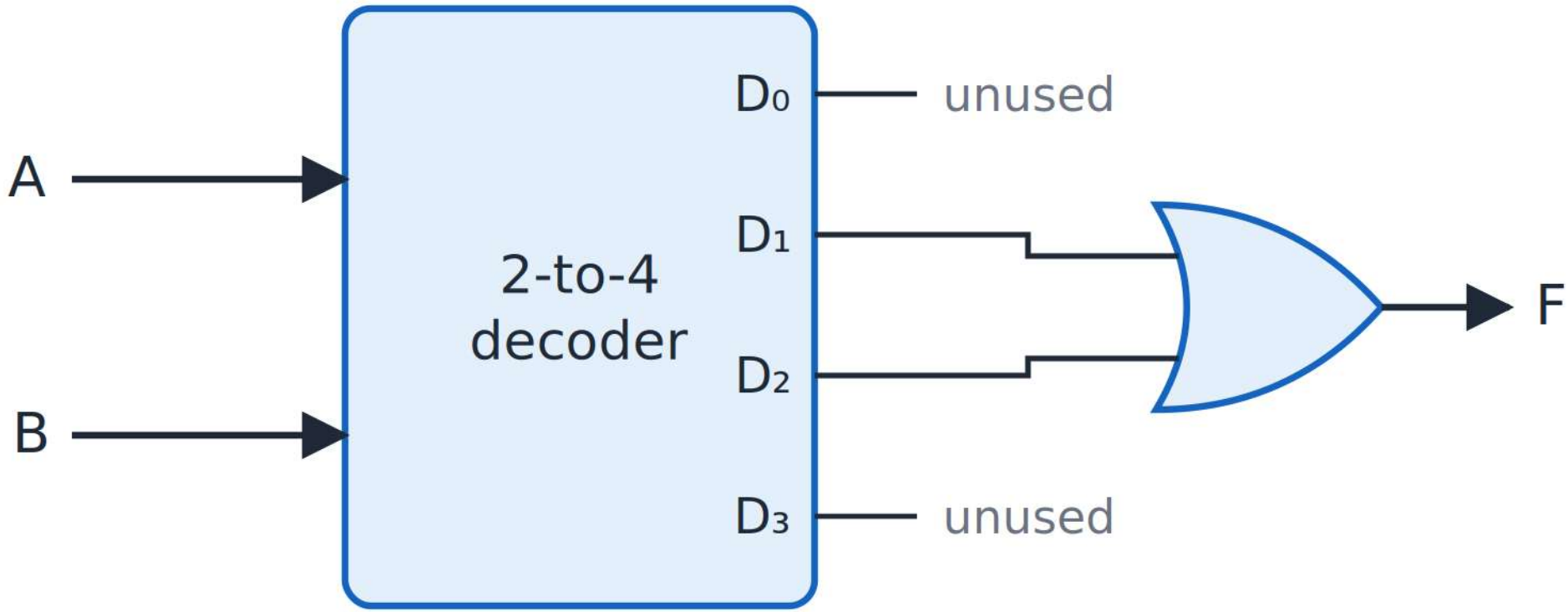


2-to-4 decoder with enable, as a block.

Decoders implement any Boolean function

- Every decoder output already is one minterm
- To implement an n-variable function: feed the n variables into an n-to-2^n decoder and **OR** the outputs for the function's minterms
- This is the canonical-SOP idea from Unit 1, built in hardware with no simplification
- **Example:** F(A,B) = Σm(1,2): $m_1 = A'B = D_1$ and $m_2 = AB' = D_2$, so $\mathbf{F = D_1 + D_2}$
- One 2-input OR gate is enough; no separate AND-gate array is needed

F = Σm(1,2) from a decoder



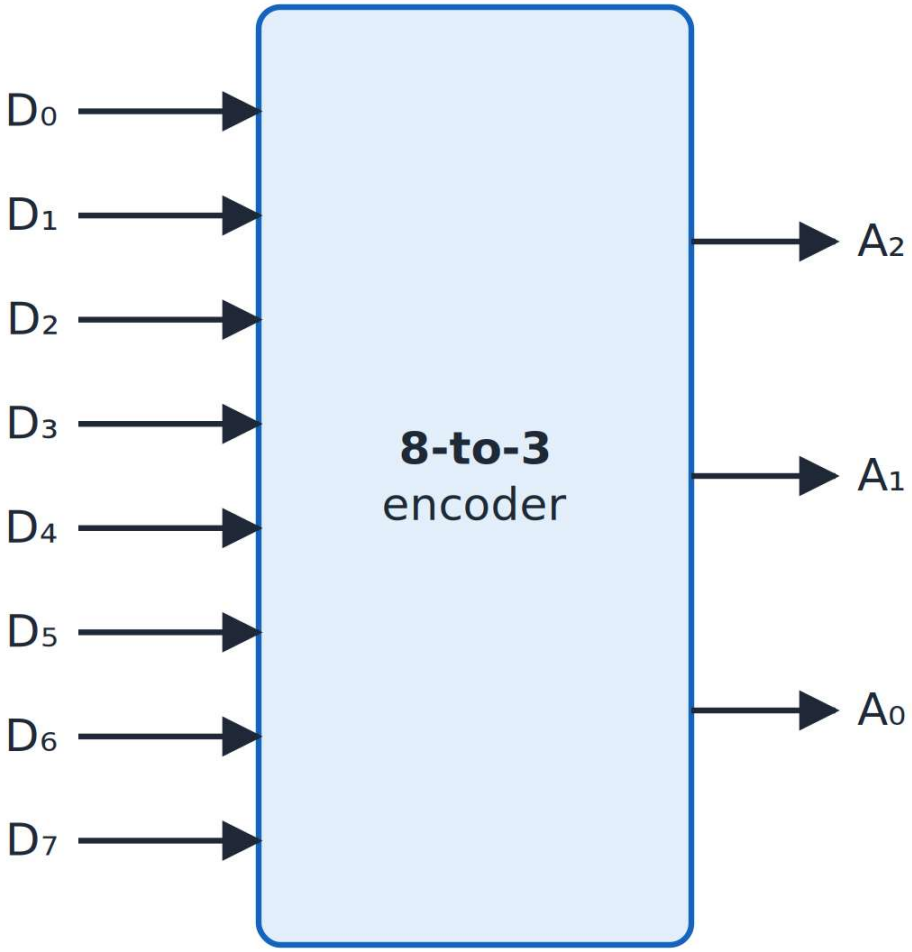
F = Σm(1,2) = $D_1 + D_2$ from a 2-to-4 decoder and one OR gate.

8-to-3 (octal-to-binary) encoder


```
A0 = D1 + D3 + D5 + D7     (inputs whose index has bit 0 set)
A1 = D2 + D3 + D6 + D7     (inputs whose index has bit 1 set)
A2 = D4 + D5 + D6 + D7     (inputs whose index has bit 2 set)
```

- An **encoder** does the reverse job: 2^n inputs (only one active at a time) go in, and n outputs carry that input's binary index
- Each output is an OR of the four inputs whose index has that bit set: three 4-input OR gates, no AND gates, no K-maps

8-to-3 encoder



- Eight one-hot inputs D_0–D_7; three outputs $A_2A_1A_0$
- Each output is an OR of the four inputs whose index has that bit set
- $A_0 = D_1 + D_3 + D_5 + D_7$
- A plain encoder needs only three 4-input OR gates

8-to-3 encoder: $A_2A_1A_0$ give the number of the active input.

Priority encoder

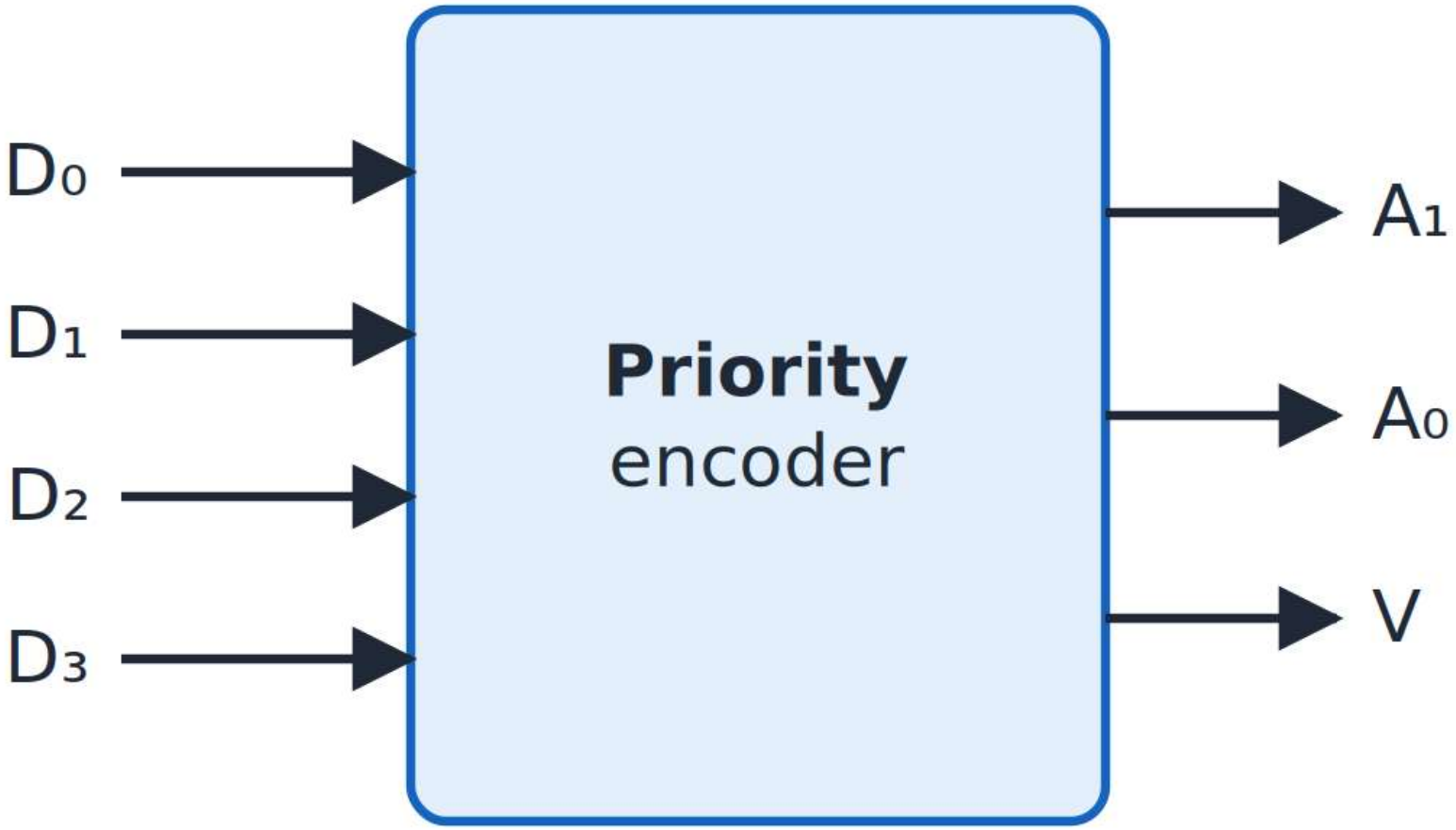
- A plain encoder assumes exactly **one** input is active; real systems cannot guarantee that
- A **priority encoder** always encodes the **highest-indexed** active input and ignores lower ones
- The X entries are don't-cares: once $D_3 = 1$, D_2, D_1 and D_0 do not matter
- **V** is a valid-output flag: 0 only when no input is active
- V is needed because $A_1 = A_0 = 0$ is otherwise ambiguous: "D_0 active" or "nothing active"?

4-input priority encoder

X means don’t-care. The highest active input always wins.

D_3	D_2	D_1	D_0	A_1	A_0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

Four-input priority encoder



Four-input priority encoder: A_1A_0 name the highest-numbered active input; V = 1 when any input is active.

3.6 Multiplexers & Demultiplexers

IN SIMPLE TERMS

A multiplexer is a selector switch from many inputs to one output; a demultiplexer sends one input to one of many outputs.

4-to-1 multiplexer

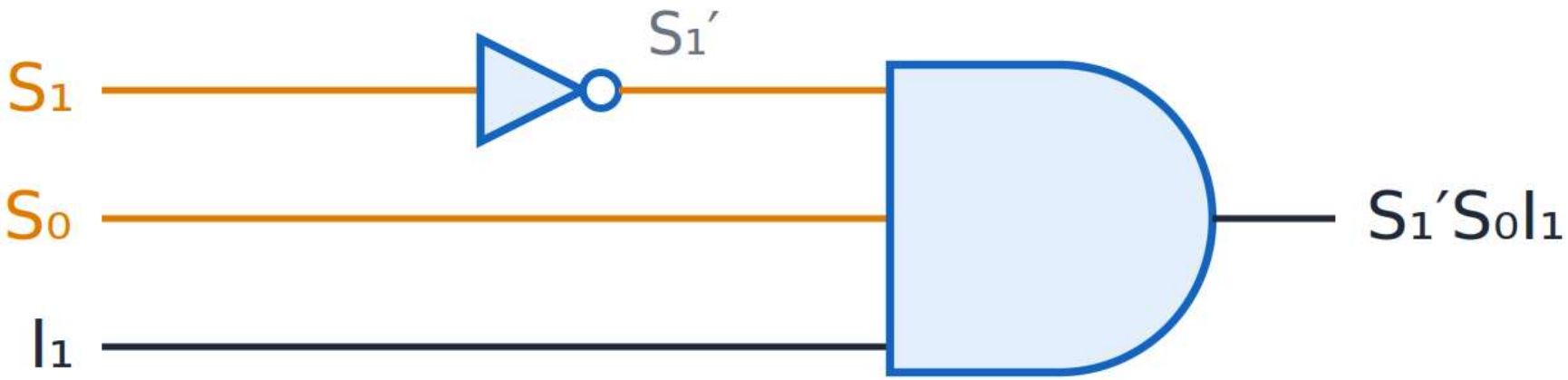
A **4-to-1 MUX** has four data inputs I_0–I_3, two select lines S_1, S_0 and one output Y. **$Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$**

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

A MUX is a decoder with data lines

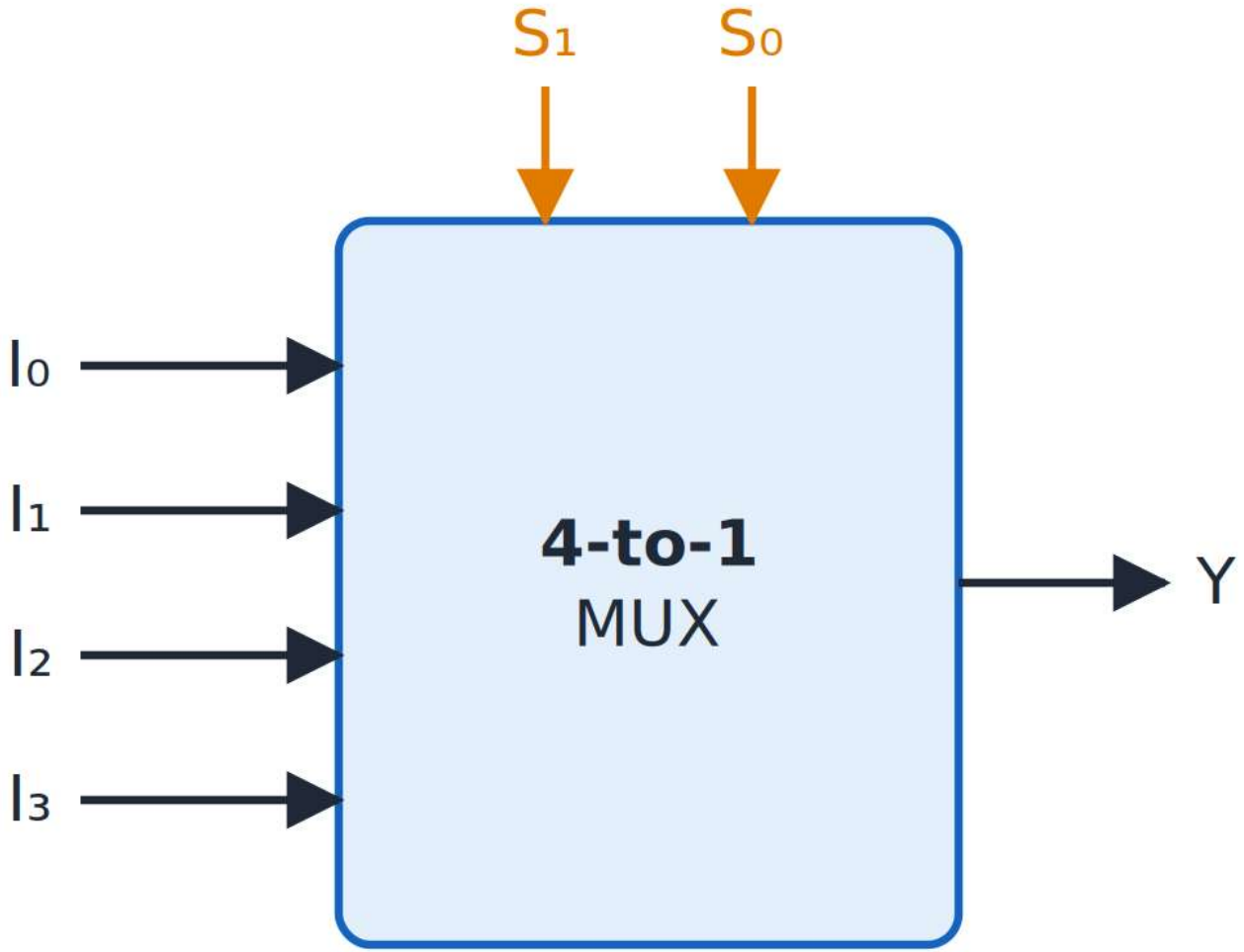
- The four terms $S_1'S_0'$, $S_1'S_0$, S_1S_0' and S_1S_0 are exactly the four outputs of a 2-to-4 decoder
- A MUX is essentially a decoder in which every output gate's third input is a **data line** instead of a shared enable
- The next slide shows the I_1 term, $S_1'S_0I_1$, built out explicitly
- The other three terms are identical, and all four feed one 4-input OR gate to produce Y

One multiplexer term



One multiplexer term: $S_1'S_0I_1$.

4-to-1 multiplexer



4-to-1 multiplexer as a block.

A MUX as a universal function generator

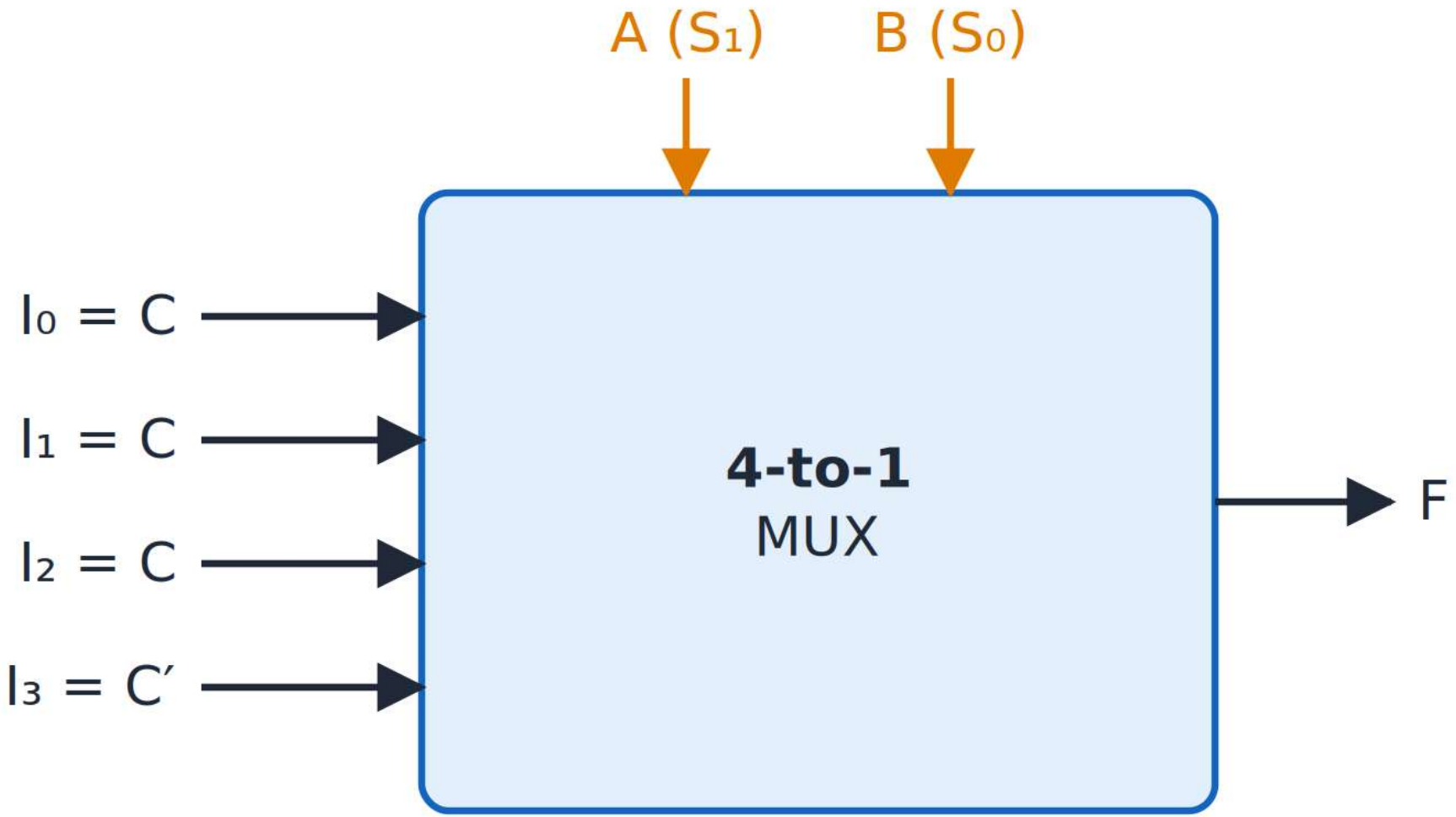
- A **2^n-to-1 MUX** can implement any function of **n + 1** variables
- Use n of the variables as select lines
- Wire each data input to 0, 1, the last variable or its complement
- Decide each input by checking what the function reduces to once the select variables are fixed
- No K-map, no separate AND/OR array and no minimisation is needed

Worked example: F(A,B,C) = Σm(1,3,5,6) on a 4-to-1 MUX


```
Select lines: S₁ = A, S₀ = B
AB = 00 (m0, m1):  F(0,0,0)=0, F(0,0,1)=1  →  F = C   →  I0 = C
AB = 01 (m2, m3):  F(0,1,0)=0, F(0,1,1)=1  →  F = C   →  I1 = C
AB = 10 (m4, m5):  F(1,0,0)=0, F(1,0,1)=1  →  F = C   →  I2 = C
AB = 11 (m6, m7):  F(1,1,0)=1, F(1,1,1)=0  →  F = C'  →  I3 = C'
```

- Wire $I_0 = I_1 = I_2 = C$ and $I_3 = C'$

F = Σm(1,3,5,6) on a multiplexer

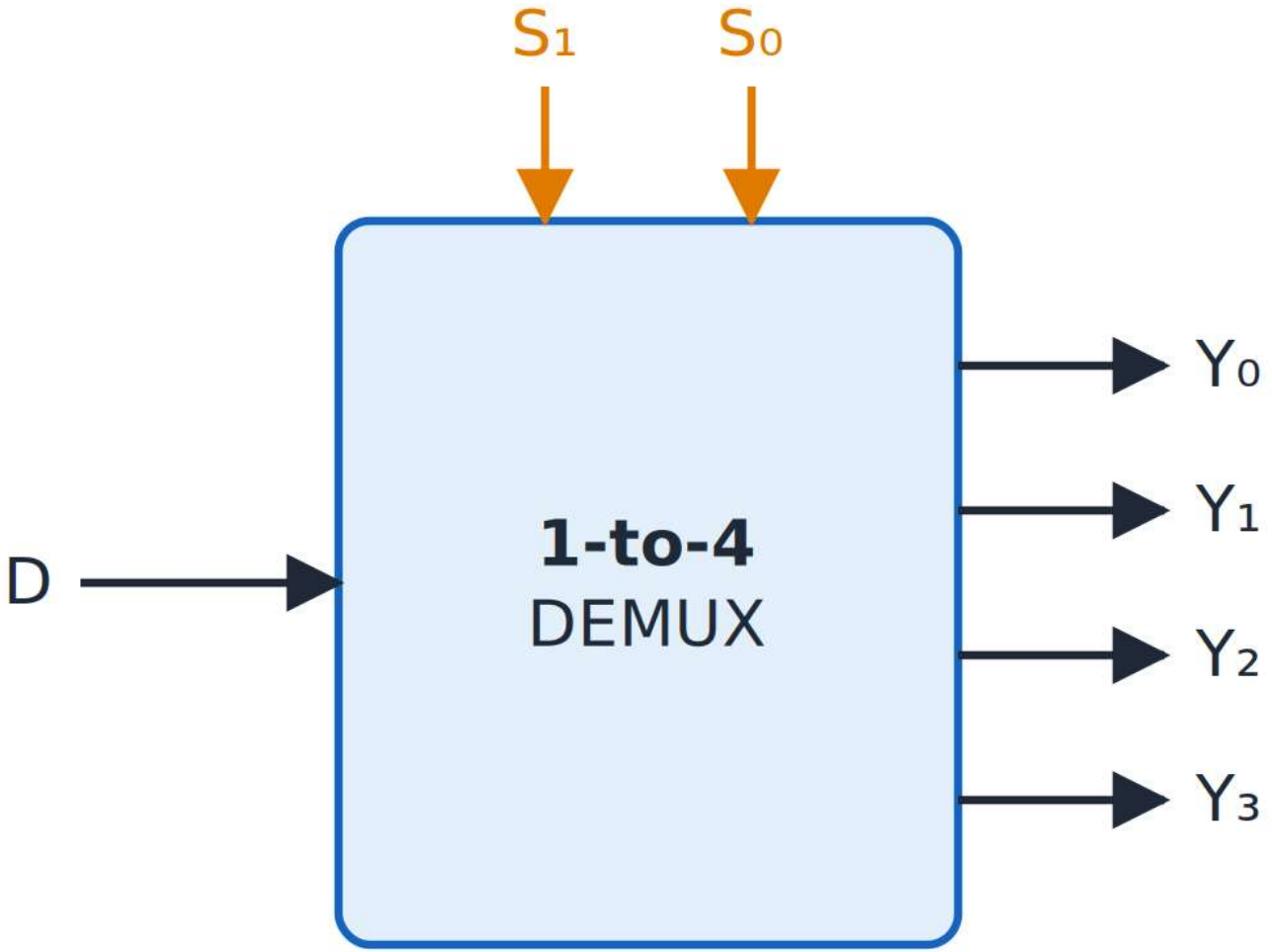


F = Σm(1,3,5,6) on a 4-to-1 multiplexer: $I_0 = I_1 = I_2 = C$, $I_3 = C'$.

Demultiplexer (data distributor)

- A **1-to-4 DEMUX** takes one data input D and two select lines S_1, S_0
- It routes D to exactly one of four outputs Y_0–Y_3; all other outputs stay 0
- $Y_0 = S_1'S_0'D$ $Y_1 = S_1'S_0D$ $Y_2 = S_1S_0'D$ $Y_3 = S_1S_0D$
- A demultiplexer **is** a decoder: the data line D takes the place of the shared enable input
- Any decoder with an enable pin can be used as a demultiplexer, and vice versa

1-to-4 demultiplexer



1-to-4 demultiplexer: D appears on the output chosen by S_1S_0.

Unit 3 summary

- **3.1 Analysis and design:** label wires, write expressions; specification → truth table → minimal function → circuit
- **3.2 Adder and subtractor:** half adder, full adder, ripple-carry chain; XOR per bit and M make an adder-subtractor
- **3.3 BCD adder and multiplier:** add 6 when $Z = C + K_3K_2 + K_3K_1 = 1$; AND gates give partial products
- **3.4 Magnitude comparator:** compare from the MSB; cascade (GT, EQ, LT) through 1-bit stages
- **3.5 Decoders and encoders:** decoder outputs are minterms; priority encoder picks the highest active input
- **3.6 Multiplexers and demultiplexers:** select one of many inputs; a DEMUX is a decoder with D as enable

Review questions (1 of 2)

- **Q1.** List the steps of the analysis procedure for a combinational circuit
- **Q2.** Derive the full adder equations S and C_{out} and show how two half adders and an OR gate build it
- **Q3.** In a 4-bit ripple-carry adder, why is the most significant sum bit valid last?
- **Q4.** How does one control line M turn an adder into an adder-subtractor?

Review questions (2 of 2)

- **Q5.** Add 8 and 7 in BCD and show when the correction of 0110 is applied
- **Q6.** Why must a magnitude comparator start at the most significant bit?
- **Q7.** Implement F(A,B) = Σm(1,2) with a 2-to-4 decoder and one gate
- **Q8.** Implement F(A,B,C) = Σm(1,3,5,6) on a 4-to-1 multiplexer

References

- **Textbook:** M. Morris Mano and Michael D. Ciletti, Digital Design
- **Adapted figures:**
- Full adder from two half adders and an OR gate (Fig. 4.8)
- Four-bit ripple-carry adder (Fig. 4.9)
- Four-bit adder-subtractor (Fig. 4.13)
- BCD adder (Fig. 4.14)
- Two-bit by two-bit binary multiplier (Fig. 4.15)